WHAT IS THE BISECTION METHOD?

Intuition, root-finding concepts, and Bolzano's Theorem

1. The Big Idea: Binary Search for Continuous Math

• The Problem: In engineering and science, equations like 2x + sin(x) = 0 or x*e^x = 2 cannot be solved with simple algebra. We must find the roots numerically.

• The Analogy: Think of the classic High-Low number guessing game. If someone thinks of a number between 1 and 100, your best first guess is 50. If they say 'too high', you know it's in [1, 50]. The Bisection Method applies this exact halving strategy to continuous mathematical functions!

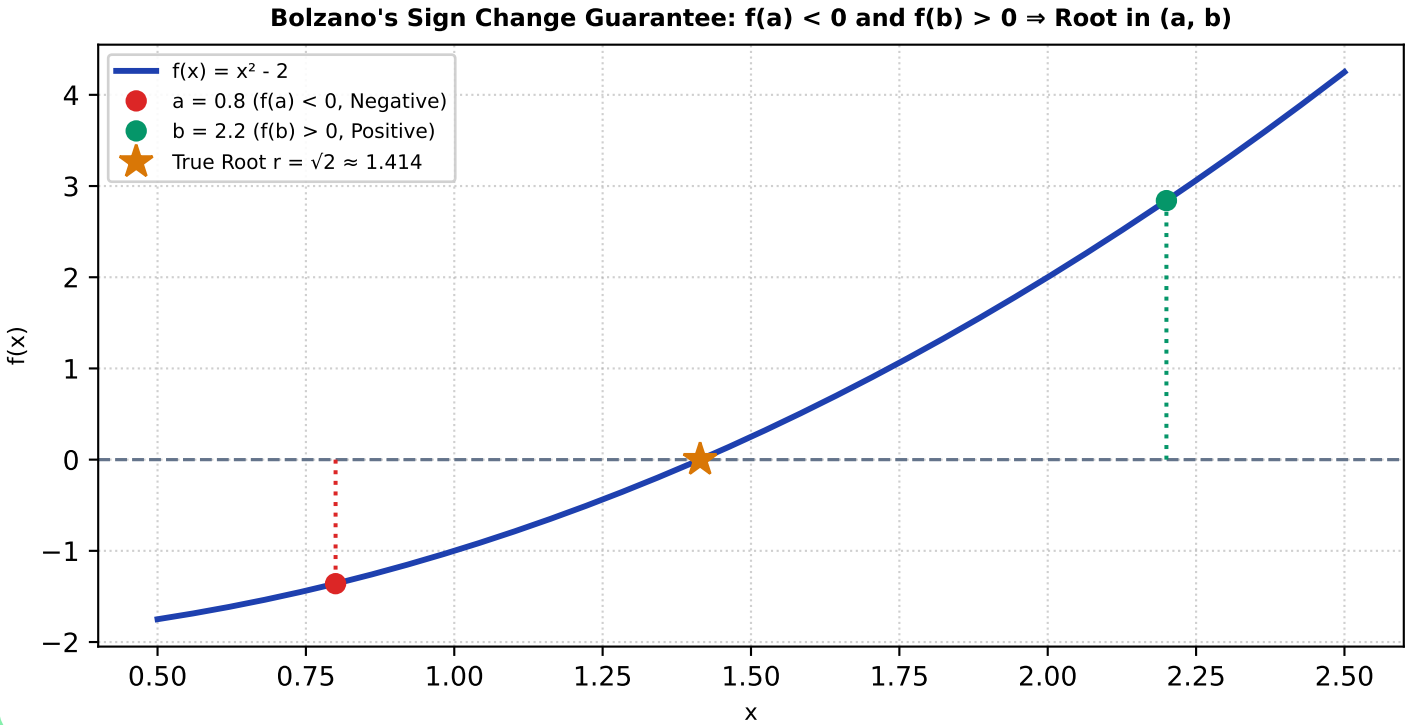
• A 'Root' (or zero) is simply the x-value where the curve crosses the horizontal axis: f(x) = 0.

2. Theoretical Guarantee: Bolzano's Theorem

If a continuous function f(x) has opposite signs at endpoints a and b:

$$f(a) \cdot f(b) < 0$$

then the curve MUST cross the horizontal axis at least once between a and b.
A root is 100% guaranteed to exist inside the bracket [a, b]!



Key Takeaway for Bisection:

Whenever you can trap a root between a positive point and a negative point,
the Bisection Method will never fail — it is globally convergent for all continuous functions.

HOW THE ALGORITHM WORKS

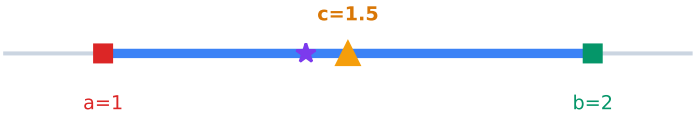
The 4-step interval halving cycle step-by-step

1. The 4-Step Iteration Loop

- **Step 1: Midpoint:** Calculate the center of the current bracket: c = (a + b) / 2
- **Step 2: Evaluate:** Compute f(c). If f(c) == 0 (or |f(c)| < 1e-15), c is an exact root!
- **Step 3: Contract:** Check sign of f(a) · f(c):
  - If f(a) · f(c) < 0: The root is in the LEFT half. Replace b = c.
  - If f(a) · f(c) > 0: The root is in the RIGHT half. Replace a = c.
- **Step 4: Check Error:** If Relative Approximate Error ea = |c_curr - c_prev| / |c_curr| ≤ tol: STOP. Else repeat!

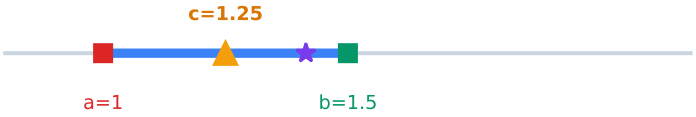
2. Visual Walkthrough: Solving $x^2 - 2 = 0$ on [1, 2]

Iteration 1



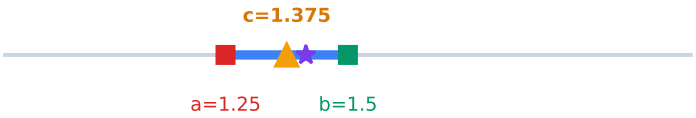
f(1)=-1 < 0, f(2)=+2 > 0
Midpoint c_1 = 1.5 (f(1.5)=+0.25 > 0)
Opposite signs on [1, 1.5] -> b = 1.5

Iteration 2



New bracket [1, 1.5] (width cut to 0.5)
Midpoint c_2 = 1.25 (f(1.25)=-0.4375 < 0)
Opposite signs on [1.25, 1.5] -> a = 1.25

Iteration 3



New bracket [1.25, 1.5] (width cut to 0.25)
Midpoint c_3 = 1.375 (f(1.375)=-0.109 < 0)
Homing closer to √2 ≈ 1.4142...

3. Why Interval Halving is Remarkable

- **Predictable Convergence: Each step halves the interval width: Width_n = (b - a) / 2^n**
- In just 10 iterations, the interval shrinks by 2^10 = 1,024 (over 1,000x smaller!).
- In 20 iterations, the interval shrinks by 2^20 = 1,048,576 (over one million times smaller!).
- **Guaranteed: Unlike Newton's method, it never diverges or shoots off if a root is bracketed.**

THE 4 TYPES OF NUMERICAL ERRORS

Definitions, exact vs approximate, and analysis roles

1. Definitions & Mathematical Formulas

- **Relative Approx Error (ea):** `|c_curr - c_prev| / |c_curr|`

  The sole convergence stopping criterion! Halts as soon as ea <= tolerance.
  Normalizes the step shift relative to the current estimate for scale-invariance.

- **Absolute Approx Error (Ea):** `|c_curr - c_prev|`

  Analysis Tool: The raw step shift between consecutive midpoints. Always computable.

- **Absolute Exact Error (Et):** `|x_true - c|`

  Analysis Tool: The true physical distance to the exact root. Reported in the summary.

- **Relative Exact Error (et):** `|x_true - c| / |x_true|`

  Analysis Tool: The exact error normalized by the true root. (Undefined if true root = 0).

2. Architecture: Stopping Criterion vs Analysis Tools

Why Relative Approximate Error is the Stopping Criterion while others are Analysis Tools:

- **Scale-Invariant Stopping (ea <= tol):**

  Whether finding a tiny root like r = 0.0012 or an astronomical one like r = 1,200,000,
  a relative tolerance (e.g. 0.0001%) guarantees 6 significant digits of precision without
  requiring manual rescaling of the tolerance.

- **The Role of the Analysis Tools (Ea, Et, et):**

  Approximate Error (Ea) shows raw step convergence in the table.
  Exact errors (Et, et) give instant benchmarking against analytical solutions whenever
  the true root is provided in the optional prompt!

3. Summary: Error Comparison Matrix

Error Type	Role in App	Requires Exact Root?	Key Benefit
Relative Approx (ea)	Stopping Criterion	No	Scale-invariant precision across all problem domains.
Absolute Approx (Ea)	Analysis Tool (Table)	No	Direct measurement of midpoint movement step size.
Absolute Exact (Et)	Analysis Tool (Summary)	Yes (Optional)	True benchmark distance to analytical solution.
Relative Exact (et)	Analysis Tool (Summary)	Yes (Optional)	Percentage accuracy relative to the true answer.